

WIPRO PYTHON AUTOMATION

REPORT-3

Module 3: Python BDD Restful

Name: Debanjali De

Enrollment Number: 12023002001076

Year: 4th Year

Branch: Computer Science and Engineering

Wipro Training Programme, 2026

TABLE OF CONTENTS

1. Introduction
2. Module 3 – Python BDD Restful
 - 2.1 Python Basics
 - 2.2 Python HTTP Libraries
 - 2.3 API Automation
 - 2.4 Handling HTTP Responses
 - 2.5 BDD
3. Module Summary
4. Learning Outcomes
5. Conclusion

1. Introduction

Behavior-Driven Development (BDD) is a software development and testing approach that describes system behavior in a form that can be understood by both technical and non-technical stakeholders. In Python automation, BDD can be used with frameworks such as Behave to express scenarios in a readable Given-When-Then style.

RESTful API automation focuses on validating web services through HTTP requests and responses. Python provides libraries and tools that make it possible to send requests, manage authentication and sessions, validate response data, and integrate API tests into a BDD framework.

This module covers Python basics required for RESTful automation, Python HTTP libraries, API automation, HTTP response handling, and the use of BDD concepts for creating maintainable API automation frameworks.

2. Module 3 – Python BDD Restful

2.1 Python Basics

This section introduces the basic technologies required before developing a Python-based BDD RESTful automation framework.

REST vs SOAP

REST (Representational State Transfer) is an architectural style commonly used for web APIs. SOAP (Simple Object Access Protocol) is a protocol that uses structured XML-based messaging. Both can be used for service communication, but they differ in architecture, message format, and typical usage.

Aspect	REST	SOAP
Nature	Architectural style	Messaging protocol
Common formats	JSON, XML, text and others	Primarily XML
Transport	Commonly HTTP/HTTPS	Can work over multiple protocols,

		commonly HTTP/HTTPS
Communication	Uses HTTP methods such as GET, POST, PUT and DELETE	Uses structured SOAP messages
Typical focus	Lightweight web APIs and resource-oriented services	Formal, contract-oriented enterprise services

Python Requests Library

The Python Requests library is a commonly used HTTP client library for sending HTTP requests from Python. It simplifies operations such as GET and POST requests, passing parameters or payloads, sending headers, handling cookies, and reading responses.

Python and Behave Setup

Behave is a Python BDD framework that allows scenarios to be written in natural-language Gherkin syntax. A typical setup contains feature files describing behavior and Python step definitions that implement those scenarios.

BDD component	Purpose
Feature file	Describes business behavior and scenarios using Gherkin.
Scenario	Represents a specific behavior or test condition.
Step	A Given, When, or Then statement in a scenario.
Step definition	Python implementation corresponding to a feature step.
Environment/hooks	Provides setup and teardown around BDD execution.

2.2 Python HTTP Libraries

Understanding GET HTTP Request Calls and Getting Response Using JSON Method

An HTTP GET request is used to retrieve a resource from a server. The server returns an HTTP response containing a status code, headers, and usually a response body. When an API returns JSON, Python can parse the response into Python data structures for validation and further processing.

Basic Features of Requests

The Requests library supports common HTTP operations and provides convenient handling of request parameters, headers, payloads, authentication, cookies, sessions, and response data.

Feature	Purpose
GET	Retrieve data from an API.
POST	Submit data or create a resource.
PUT	Replace or update a resource.
PATCH	Partially update a resource.
DELETE	Remove a resource.
Headers	Send metadata such as content type or authorization information.
Parameters	Pass query-string values to a request.

Installation and Quick Start

The HTTP library is installed in the Python environment used for the automation project. After installation, a client can construct a request, send it to an API endpoint, receive the response, and inspect its status, headers, and body.

Validating Response Status Codes and Headers Using Response Object

The response object provides information required for validation. Status-code validation checks whether the server returned the expected HTTP result, while header validation checks response metadata such as content type.

Automating POST HTTP Request with Payload and Headers

A POST request is commonly used to submit data to a service. The request may contain a JSON payload and headers that describe the request or provide authorization. Automation verifies both the HTTP result and the returned response content.

End-to-End Automation Flow of API Calls Using Python

An end-to-end API automation flow generally includes preparing test data, configuring the endpoint, sending the request, validating the response status and headers, checking response content, and recording the result.

Step	Activity
1	Prepare endpoint, data and configuration.
2	Build the HTTP request.
3	Send the request to the API.
4	Receive and inspect the response.
5	Validate status, headers and body.
6	Report or store the test result.

2.3 API Automation

Setting Global Configurations Using Python Config Object

Global configuration stores values that may be reused by many API tests, such as base URLs, environment settings, timeouts, or other common parameters. Centralizing configuration avoids hard-coding the same values throughout the framework.

Optimizing Resources and Payload from Externally Reusable Data

Externalizing request payloads and reusable data separates test data from automation logic. This makes it easier to reuse the same test flow with different input values and simplifies maintenance when data changes.

Authenticating APIs Using Python Automation Auth Method – Example

API authentication verifies that a client is permitted to access a protected service. Common approaches include credentials, API keys, tokens, and other authorization mechanisms. Automation supplies the required authentication information and validates the resulting response.

Importance of Session Managing in API Automation

Session management allows related requests to share connection information, cookies, and other session state. It is useful when multiple API calls belong to the same user or workflow.

Send and Manage Cookies for API Request Calls

Cookies are pieces of state information sent between a client and server. API automation may need to send, receive, store, or reuse cookies to maintain a session or reproduce a browser-like interaction.

Setting Up MySQL Instance to Drive the Data from Python Code

Database-driven automation uses a database as a source of test data or expected results. Python can connect to a MySQL instance through an appropriate database connector, execute queries, retrieve records, and use the results in automation.

Setup Database with Example Table Data and Start Connection

A database-driven test begins by preparing the required database and table data and establishing a connection from Python. The retrieved records can then be supplied to API test scenarios.

PUT, PATCH, and DELETE Methods

PUT is generally used to replace or update a resource, PATCH is used for partial updates, and DELETE is used to remove a resource. API automation verifies the status code and response data for each operation.

Method	Typical purpose
PUT	Replace or update a resource representation.
PATCH	Apply a partial update to a resource.
DELETE	Delete a resource.

Updating and Deleting Data via API

Automation can validate update and delete operations by first identifying a resource, sending the appropriate request, checking the response, and confirming the resulting resource state.

Differences Between PUT and PATCH

Aspect	PUT	PATCH
Update scope	Generally represents a complete replacement/update	Generally represents a partial modification
Typical payload	May contain the full resource representation	Usually contains only the fields to change
Use case	Replace/update a resource	Modify selected resource fields

2.4 Handling HTTP Responses

Understanding Response Objects

A response object represents the server's reply to an HTTP request. It typically provides access to the status code, response headers, body/content, cookies, and other response information.

Reading Response Content

Response content can be read in formats such as text or JSON. JSON response content can be converted into Python dictionaries and lists for structured validation.

Handling Binary and Text Data

Text responses contain character data that can be read and processed as text. Binary responses contain raw bytes and may represent files or other non-text content. Automation should use the appropriate representation when validating each type.

Status Codes and Error Handling

HTTP status codes communicate the result of a request. API automation should validate expected status codes and handle unexpected responses in a controlled way.

Category	Meaning	Examples
2xx	Successful request	200 OK, 201 Created
3xx	Redirection	301, 302
4xx	Client-side error	400, 401, 403, 404
5xx	Server-side error	500, 502, 503

Success and Error Codes

Success codes indicate that the request was processed successfully, while error codes indicate a problem with the request, authorization, resource, or server. The expected status code should be part of the test validation.

Exception Handling in Requests

Exception handling prevents an unexpected network, timeout, connection, or request-processing problem from causing uncontrolled test termination. Appropriate exception handling also helps produce meaningful test results.

Advanced Response Handling

Advanced response handling may include validating nested JSON structures, checking multiple headers, extracting values for subsequent requests, comparing response data with expected data, and handling different response formats.

Streaming Large Responses

Large responses can be processed as streams instead of loading the complete content into memory at once. This approach can reduce memory usage when dealing with large response bodies or files.

Handling Cookies and Sessions

Cookies and sessions allow state to be maintained across related HTTP requests. Automation should preserve the required session information when a workflow depends on authentication or server-side state.

2.5 BDD

What are the BDD Tools We Use to Automate in Python Language

BDD automation tools provide a readable way to describe expected system behavior and connect those descriptions to executable automation code. In this module, Behave is the primary Python BDD framework used for feature files, scenarios, step definitions, and hooks.

BDD element/tool	Purpose
Behave	Python BDD framework for executing Gherkin scenarios.
Gherkin	Human-readable language used to describe behavior.
Feature file	Stores features and scenarios.
Step definitions	Python implementations of Gherkin steps.
Hooks	Run setup/cleanup around scenarios or test execution.
Allure	Can be used to generate detailed test execution reports.

Create Step Definition File with Auto-Generated Syntaxes and Annotations

Step definitions connect Gherkin steps to executable Python functions. A step-definition file contains the implementation for Given, When, and Then statements. The annotations or decorators associate a step pattern with its Python implementation.

Implement API Automation Actual Logic into Step Definitions

The API automation logic can be implemented inside the functions associated with BDD steps. A scenario can therefore describe the behavior in Gherkin while the step definitions perform the HTTP requests and validations.

Implement Hooks for the Scenarios by Defining environment.py File

Behave hooks provide lifecycle points for preparing and cleaning the test environment. The environment.py file can define hooks that execute before or after features, scenarios, or steps, depending on the framework configuration.

Implement Parameterization to the Test Scenarios Using Example Outline Keyword

Scenario Outline allows the same scenario structure to be executed with multiple sets of input data. The Examples section provides the values that are substituted into the scenario parameters.

Use Tagging Mechanism to Run Selected Tests in Framework

Tags are labels attached to features or scenarios. They can be used to organize tests and selectively run specific groups, such as smoke tests, regression tests, API tests, or environment-specific scenarios.

Best Practices to Write Reusable Step Definitions Code for API Test Scenarios

- Keep step definitions focused on behavior rather than duplicating low-level implementation.
- Reuse common request, validation, and data-handling functions.
- Separate configuration and test data from step-definition logic.
- Use meaningful scenario and step names.
- Avoid unnecessary duplication between related scenarios.
- Keep API-specific helper functions reusable across scenarios.

Integrate Tagging and Hooks to Build Generalized BDD Code for Python API Tests

Combining tags and hooks creates a reusable framework structure. Hooks can perform common setup and cleanup, while tags can control which scenarios are executed. This makes it possible to use the same framework for different categories of API tests.

Generate Allure Reports for Python API BDD Framework

Allure is a reporting tool that can present test execution results in a structured and interactive format. When integrated with a Python BDD framework, it can help display scenario results, steps, failures, and other execution information for easier analysis.

3. Module Summary

Module 3 combines Python, RESTful API automation, HTTP response handling, and BDD. It begins with REST versus SOAP, the Python Requests library, and Behave setup. It then covers HTTP methods and response validation, configuration, authentication, sessions, cookies, database-driven data, and response handling. Finally, it introduces Behave BDD concepts such as step definitions, hooks, Scenario Outline, tagging, reusable code, and Allure reporting.

Section	Core focus
2.1 Python Basics	REST vs SOAP, Requests library, Python/Behave setup.
2.2 Python HTTP Libraries	HTTP requests, response validation, payloads, headers, end-to-end API flow.
2.3 API Automation	Configuration, reusable data, authentication, sessions, cookies, database data, HTTP methods.
2.4 Handling HTTP Responses	Response objects, content, binary/text data, status/error handling, streaming, sessions.
2.5 BDD	Behave, step definitions, hooks, Scenario Outline, tags, reusable steps, Allure reporting.

4. Learning Outcomes

- Explain the difference between REST and SOAP and identify their common characteristics.
- Understand the purpose and basic features of the Python Requests library.
- Explain GET, POST, PUT, PATCH, and DELETE operations in API automation.
- Validate response status codes, headers, response content, cookies, and sessions.
- Understand authentication, configuration management, reusable external data, and database-driven API testing.

- Explain HTTP response handling, exception handling, binary/text content, and streaming responses.
- Define BDD and explain the role of Behave, Gherkin, feature files, scenarios, and step definitions.
- Understand hooks, Scenario Outline parameterization, tagging, reusable step definitions, and generalized BDD frameworks.
- Understand the purpose of Allure reporting in a Python API BDD automation framework.

5. Conclusion

Python BDD Restful automation combines API testing capabilities with behavior-driven development. Python HTTP libraries make it possible to communicate with RESTful services, validate responses, manage authentication and sessions, and work with external or database-driven data. Behave adds a readable BDD layer through feature files, step definitions, hooks, parameterization, and tagging. Together, these concepts provide a structured foundation for building maintainable Python REST API automation frameworks.